



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش آزمایش ۹ : پیاده سازی حافظه های شرکت پذیر نوع سه گانه

اعضای گروه

امیرشایان سلیمانی نیا

حسن پناهی

امین استادکاظمی

نام درس

آز طراحی سیستم های دیجیتال

تابستان ۱۴۰۵

نام استاد درس

دکتر انصاری

فهرست مطالب

۱	مقدمه و مبانی نظری	۲
۲	شرح معماری و نحوه پیاده‌سازی	۲
۳	شماتیک RTL و سنتز مدار	۳
۴	شبیه‌سازی و راستی‌آزمایی	۴
۵	نتیجه‌گیری	۵

۱ مقدمه و مبانی نظری

حافظه‌های محتوا آدرس پذیر یا CAM (Content Addressable Memory) دسته‌ای خاص از حافظه‌های سخت‌افزاری هستند که عملکرد آن‌ها برعکس حافظه‌های معمول مبتنی بر آدرس (RAM) است. در حافظه‌های متعارف، با دادن یک آدرس مشخص، داده ذخیره‌شده در آن آدرس در خروجی قرار می‌گیرد؛ در حالی که در حافظه‌های CAM، داده مورد جستجو به عنوان ورودی داده شده و سیستم در صورت تطابق، آدرس یا خطوط تطابق (Match Lines) را در یک چرخه ساعت گزارش می‌کند. حافظه‌های شرکت پذیر نوع سه‌گانه یا TCAM (Ternary CAM) نوع پیشرفته‌تری از این معماری هستند که علاوه بر وضعیت‌های منطقی ۰ و ۱، قابلیت ذخیره حالت بی‌تفاوت یا (X) Don't Care را نیز فراهم می‌آورند. وجود حالت X به این معناست که در فرآیند مقایسه، بیت‌های مشخص شده با X همواره برابر و منطبق فرض می‌شوند. این ویژگی به ویژه در سیستم‌های مسیریابی شبکه، جداول IP Lookup بر مبنای CIDR، فایروال‌ها و پایگاه‌های داده کاربرد حیاتی دارد.

۲ شرح معماری و نحوه پیاده‌سازی

برای پیاده‌سازی حافظه TCAM با ابعاد ۱۶ ثابت ۱۶ بیتی در سخت‌افزار (FPGA)، امکان ذخیره مستقیم حالت سه‌گانه در رجیسترهای باینری وجود ندارد. بدین منظور، از معماری دوبرداری داده و ماسک (Data-Mask Architecture) استفاده شده است:

- آرایه داده (mem_data): مقادیر بیتی اصلی سطرهای حافظه (۰ یا ۱) را نگهداری می‌کند.

- آرایه ماسک (mem_mask): موقعیت بیت‌های بی‌تفاوت (X) را مشخص می‌سازد؛ به طوری که اگر بیتی از ماسک برابر با ۱ باشد، آن بیت در مقایسه بی‌اثر تلقی می‌شود.

معادله منطقی برای بررسی انطباق بیت j - ام از سطر i - ام با داده ورودی جستجو (search_data) به صورت زیر بیان می‌شود:

$$\text{Match_bit}_{i,j} = (\text{search_data}_j \oplus \text{mem_data}_{i,j}) \cdot \overline{\text{mem_mask}_{i,j}} = 0 \quad (1)$$

یا در فرم برداری باینری:

(۲)

$$\text{match_lines}[i] = ((\text{search_data} \& \sim \text{mem_mask}[i]) == (\text{mem_data}[i] \& \sim \text{mem_mask}[i]))$$

همچنین یک واحد کدگذار اولویت (Priority Encoder) برای استخراج آدرس نخستین سطر منطبق و یک گیت OR تجمیعی (Reduction OR) جهت تولید پرچم match_found پیاده‌سازی گردید.

۱-۲ کد سخت‌افزاری ماژول (Verilog HDL) و تشریح عملکرد

کد پیاده‌سازی شده ماژول tcam_16_16 در ادامه آورده شده است:

```
1 module tcam_16_16 (  
2     input wire      clk,  
3     input wire      rst,  
4     input wire      wr_en,  
5     input wire [3:0] wr_addr,  
6     input wire [15:0] wr_data,  
7     input wire [15:0] wr_mask,  
8     input wire      search_en,  
9     input wire [15:0] search_data,  
10    output reg [15:0] match_lines,  
11    output wire      match_found,  
12    output reg [3:0] match_addr  
13 );  
14  
15    reg [15:0] mem_data [0:15];  
16    reg [15:0] mem_mask [0:15];  
17  
18    integer i;  
19  
20    always @(posedge clk or posedge rst) begin  
21        if (rst) begin  
22            for (i = 0; i < 16; i = i + 1) begin  
23                mem_data[i] <= 16'h0000;  
24                mem_mask[i] <= 16'h0000;  
25            end  
26        end else if (wr_en) begin  
27            mem_data[wr_addr] <= wr_data;  
28            mem_mask[wr_addr] <= wr_mask;  
29        end  
30    end  
31  
32    always @(*) begin  
33        if (search_en) begin  
34            for (i = 0; i < 16; i = i + 1) begin  
35                match_lines[i] = ((search_data & ~mem_mask[i]) ==  
36                    (mem_data[i] & ~mem_mask[i]));  
37            end  
38        end else begin  
39            match_lines = 16'h0000;  
40        end  
41    end
```

```

41
42 assign match_found = |match_lines;
43
44 always @(*) begin
45     match_addr = 4'd0;
46     for (i = 15; i >= 0; i = i - 1) begin
47         if (match_lines[i]) begin
48             match_addr = i[3:0];
49         end
50     end
51 end
52
53 endmodule

```

در پیاده‌سازی فوق، عملکرد مدار به چهار بخش کلیدی تقسیم شده است:

۱. **حافظه ترتیبی (Sequential Memory Write):** دو آرایه ۱۶ بیتی از ثبات‌ها برای ذخیره داده و ماسک تعریف شده‌اند. در لبه بالارونده کلاک و در صورت فعال بودن سیگنال `wr_en`، داده و ماسک متناظر در آدرس مشخص شده توسط `wr_addr` ذخیره می‌شوند. سیگنال `rst` نیز به صورت ناهمگام تمام ثبات‌ها را صفر می‌کند.

۲. **مقایسه‌کننده موازی ترکیبی (Parallel Combinational Match):** این بخش قلب تپنده TCAM است؛ با فعال بودن `search_en`، عملگر بیت‌به‌بیت Bitwise AND داده جستجو و محتوای ثبات را با معکوس ماسک (`~mem_mask`) ترکیب می‌کند. به این ترتیب بیت‌هایی که ماسک آن‌ها ۱ است نادیده گرفته شده و در صورت برابری سایر بیت‌ها، بیت متناظر در بردار ۱۶ بیتی `match_lines` برابر با ۱ قرار می‌گیرد.

۳. **سیگنال تطابق کلی (match_found):** با استفاده از عملگر کاهش‌یابنده Reduction OR (`|match_lines`)، در صورتی که حداقل یکی از خطوط تطابق یافته باشد، این پرچم مقدار ۱ به خود می‌گیرد.

۴. **کدگذار اولویت (Priority Encoder):** حلقه‌ای ترکیبی از بالاترین شماره سطر تا پایین‌ترین سطر بررسی می‌کند و در نهایت کمترین شاخص عددی (بالاترین اولویت) که تطابق داشته باشد را به عنوان `match_addr` در خروجی قرار می‌دهد.

۳ سنتز مدار

طراحی ماژول در نرم افزار Intel Quartus Prime سنتز گردید. مدار شامل یک بخش ترتیبی برای نوشتن سنکرون در حافظه با سیگنال کلاک و ریست، و یک بخش ترکیبی گسترده از مقایسه کننده ها به منظور جستجوی تمام موازی در تمامی ۱۶ سطر است.

گزارش سنتز و مصرف منابع سخت افزاری (Flow Summary) شامل تعداد عناصر منطقی (Logic Elements)، ثبات ها و پین های مصرفی در شکل ۱ آورده شده است.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sat Sep 5 20:15:38 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	tcam_16_16
Top-level Entity Name	tcam_16_16
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	200 / 56,480 (< 1 %)
Total registers	512
Total pins	77 / 268 (29 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	0 / 156 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

شکل ۱: گزارش مصرف منابع و سنتز نهایی (Flow Summary)

۴ شبیه سازی و راستی آزمایی

به منظور ارزیابی کارایی ماژول، تست بنچی طراحی گردید که الگوهای خواسته شده در صورت مسئله را در حافظه بارگذاری می کند:

• سطر ۰: 0110XXXX_XXXXXXXX

• سطر ۱: 0X1X11X0_XXXXXXXX

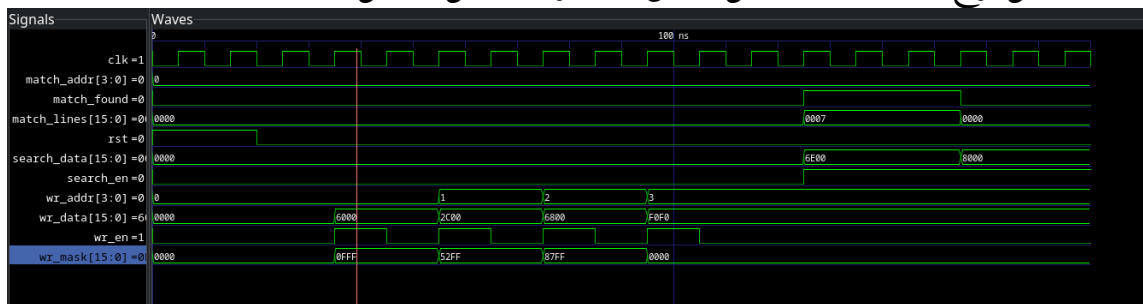
• سطر ۲: X1101XXX_XXXXXXXX

کد و فرآیند آماده‌سازی تست‌بنچ در شکل ۲ نشان داده شده است.

```
> vvp tcam sim
VCD info: dumpfile tcam_waveform.vcd opened for output.
Match Found? : 1
Match Lines : 00000000000000111
First Match : Row 0
Match Found? : 0
Match Lines : 00000000000000000
tb_tcam_16_16.v:90: $finish called at 180000 (1ps)
```

شکل ۲: بخشی از کد تست‌بنچ جهت مقداردهی الگوها و جستجو

با اعمال بردار ورودی 01101110_00000000 در سیگنال جستجو، هر سه سطر به صورت همزمان شرایط انطباق را احراز کرده و خروجی match_lines مقادیر بیت‌های ۰، ۱ و ۲ را برابر با ۱ نشان می‌دهد. شکل موج شبیه‌سازی حاصل از این سناریو در شکل ۳ قابل مشاهده است.



شکل ۳: شکل موج‌های شبیه‌سازی و بررسی عملکرد سیگنال‌های match_lines و match_found

۵ نتیجه‌گیری

در این آزمایش، ساختار سخت‌افزاری حافظه شرکت‌پذیر سه‌گانه (TCAM) با موفقیت به زبان Verilog توصیف، سنتز و راستی‌آزمایی شد. نتایج نشان داد که استفاده از دو آرایه مجزای دیتای باینری و بیت‌های ماسک، راهکاری کارآمد برای پیاده‌سازی منطقی حالت‌های X (Don't Care) در تراشه‌های دیجیتال فراهم می‌آورد. شبیه‌سازی تایمینگ و مقایسه‌های همزمان تایید کردند که مازول طراحی شده توانایی انطباق موازی چندگانه و تشخیص صحیح اولویت سطرها را در یک چرخه کلاک دارا می‌باشد.